

数据结构机考学习4：链表练习题总结

前面已经学过链表的基本操作了，

今天主要整理一下自己做的几道链表练习题。

这些代码是我看到题目后自己写的，

所以大概率会有更优的解法和写法，建议问一下ai看看。

一：统一说明

下面的代码使用的是**带头结点的单链表**。

也就是说：

`first` 指向头结点，

真正的第一个数据结点是：

`first->next`

结点定义：

```
template <class T>
struct Node
{
    T data;
    Node<T>* next;
};
```

注意：

如果是带头结点的链表，

打印的时候应该从 `first->next` 开始。

如果直接从 `first` 开始打印，会把头结点也打印出来。

二：题目

题目1：删除链表中的重复元素

题目：

给定一个链表，删除重复出现的元素，保留第一次出现的元素。

例如：

1 2 3 2 4 3 5

处理后变成：

1 2 3 4 5

哈希表（自己写的时候用的方法）

方法概述：

用一个哈希表记录某个值是否已经出现过。

如果当前结点的值第一次出现，

就保留它，继续向后走。

如果当前结点的值已经出现过，

就删除当前结点。

核心思路：

用两个指针：

- pre：当前结点的前驱结点
- p：当前正在判断的结点

如果 `p->data` 没出现过，

就把它加入哈希表，

然后 `pre` 和 `p` 一起后移。

如果 `p->data` 已经出现过，

就让：

```
pre->next = p->next;
```

然后删除 `p`。

代码：

```
void clear()
{
    unordered_set<T> st;

    Node<T>* pre = first;
    Node<T>* p = first->next;

    while (p != NULL)
    {
        if (st.count(p->data) == 0)
        {
            st.insert(p->data);
            pre = p;
            p = p->next;
        }
        else
        {
            Node<T>* q = p;
            pre->next = p->next;
            p = p->next;
            delete q;
        }
    }
}
```

题目2：删除有序链表中所有重复出现的元素

题目：

给定一个**有序链表**，

删除所有重复出现的元素，

只保留只出现一次的元素。

例如：

1 2 2 3 4 4 5

处理后变成：

1 3 5

再比如：

1 1 1 2 3 3

处理后变成：

2

核心思路

因为链表是有序的，

所以相同的元素一定是连续出现的。

因此可以把相同的元素看成一组。

用 p 指向当前这一组的第一个结点，

用 q 一直向后走，

找到这一组的后面第一个不同的结点。

如果这一组只有一个结点，说明这个值是独立的，保留。

如果这一组有多个结点，说明这个值重复出现过，整组删除。

方法概述：

定义三个指针：

- pre：已经处理好部分的最后一个结点
- p：当前这一组的第一个结点
- q：用来向后找这一组的结尾

循环部分：

先让 $q = p \rightarrow next$ ，

然后只要 q 不为空，

并且 $q \rightarrow data == p \rightarrow data$ ，

就继续后移。

最后分两种情况：

1. 如果没有重复，保留 p
2. 如果有重复，删除从 p 到 q 之前的所有结点

代码：

```
void clear()
{
    Node<T>* pre = first;
    Node<T>* p = first->next;

    while (p != NULL)
    {
        Node<T>* q = p->next;
        bool repeat = false;
```

```

while (q != NULL && q->data == p->data)
{
    repeat = true;
    q = q->next;
}

if (!repeat)
{
    pre = p;
    p = p->next;
}
else
{
    while (p != q)
    {
        Node<T>* del = p;
        p = p->next;
        delete del;
    }

    pre->next = q;
    p = q;
}
}
}

```

题目3：链表循环左移和右移

题目：

给定一个链表和一个整数 k ，

要求把链表循环移动 k 位。

例如链表为：

1 2 3 4 5

循环左移 2 位后：

3 4 5 1 2

循环右移 2 位后：

4 5 1 2 3

核心思路

链表循环左移 k 位，

其实就是把前 k 个结点剪下来，

接到链表尾部。

例如：

1 2 | 3 4 5

左移 2 位后：

3 4 5 | 1 2

方法概述：

1. 先处理 k

$k = k \% \text{len};$

如果 $k == 0$ ，说明不需要移动。

2. 找到链表尾结点 `tail`

3. 找到第 k 个结点 `cut`

4. 新的第一个结点是：

cut->next

5. 让尾结点接到原来的第一个结点

6. 把第 k 个结点后面断开

循环左移代码：

```
int length()
{
    int len = 0;
    Node<T>* p = first->next;

    while (p != NULL)
    {
        len++;
        p = p->next;
    }

    return len;
}

void leftShift(int k)
{
    int len = length();

    if (len <= 1) return;

    k = k % len;
    if (k == 0) return;

    Node<T>* tail = first->next;
    while (tail->next != NULL)
    {
        tail = tail->next;
    }

    Node<T>* cut = first->next;
    for (int i = 1; i < k; i++)
    {
        cut = cut->next;
    }
}
```



```
Node<T>* newFirst = cut->next;

tail->next = first->next;
cut->next = NULL;
first->next = newFirst;
}
```

循环右移代码：

右移 k 位，

可以转化成左移：

右移 k 位 = 左移 $n-k$ 位

代码如下：

```
void rightShift(int k)
{
    int len = length();

    if (len <= 1) return;

    k = k % len;
    if (k == 0) return;

    leftShift(len - k);
}
```

题目4：链表逆置

题目：

把链表反转，不新建结点，只修改指针。

例如：

1 2 3 4 5

变成：

5 4 3 2 1

核心思路

和头插法创建链表很像。先把头结点断开，

然后把原链表中的每个结点依次头插到头结点后面。

代码：

```
void reverse()
{
    Node<T>* p = first->next;
    first->next = NULL;

    while (p != NULL)
    {
        Node<T>* q = p->next;

        p->next = first->next;
        first->next = p;

        p = q;
    }
}
```

注意：

一定要先用 `q` 保存 `p->next` ，

否则修改 `p->next` 后，后面的链表就找不到了。

题目5：删除链表中所有值为 `x` 的结点

题目：

删除链表中所有值等于 `x` 的结点。

例如：

1 2 3 2 4

删除 2 后：

1 3 4

核心思路

还是用前驱指针。

pre 指向当前结点的前一个结点，

p 指向当前结点。

如果 `p->data == x` ，

就删除 p 。

否则两个指针一起后移。

代码：

```
void deleteX(T x)
{
    Node<T>* pre = first;
    Node<T>* p = first->next;

    while (p != NULL)
    {
        if (p->data == x)
        {
            Node<T>* q = p;
            pre->next = p->next;
            p = p->next;
            delete q;
        }
        else
        {
            pre = p;
            p = p->next;
        }
    }
}
```

```
    }  
  }  
}
```

注意：

删除结点时，pre 不要后移。

因为删除当前结点后，pre->next`可能还是需要继续判断的新结点。